

```
In [1]: """Imports and runtime diagnostics."""

import json
import os
import numpy as np
import pandas as pd
import random
import torch

from sklearn.model_selection import train_test_split
from transformers import AutoTokenizer, AutoModel
from torch.nn.utils.rnn import pad_sequence

print("Torch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

Torch version: 2.11.0+cu128
 CUDA available: True

```
In [2]: """
File paths
"""

from pathlib import Path

def _project_root():
    start = Path.cwd().resolve()
    for candidate in (start, *start.parents):
        if (candidate / "db").is_dir() and (candidate / "notebooks").is_dir():
            return candidate
    raise FileNotFoundError(f"Could not find project root from {start}")

ROOT = _project_root()
MODELS_DIR = ROOT / "models"
MODELS_DIR.mkdir(parents=True, exist_ok=True)

GOEMOTIONS_CHECKPOINT = str(MODELS_DIR / "best_goemotions_focal_model.pt")
CMED_CHECKPOINT = str(MODELS_DIR / "best_cmed_focal_model.pt")

GOEMOTIONS_TRAIN_PATH = str(ROOT / "db" / "goemotions" / "train.tsv")
GOEMOTIONS_VALID_PATH = str(ROOT / "db" / "goemotions" / "dev.tsv")
GOEMOTIONS_TEST_PATH = str(ROOT / "db" / "goemotions" / "test.tsv")
GOEMOTIONS_EMOTIONS_PATH = str(ROOT / "db" / "goemotions" / "emotions.txt")
GOEMOTIONS_EKMAN_PATH = str(ROOT / "db" / "goemotions" / "ekman_mapping.json")

MYBULLY_VALID_PATH = "hf://datasets/mohanrj/MYBully/val.csv"
CMED_DATA_PATH = "hf://datasets/Johnson8187/Chinese_Multi-Emotion_Dialogue_Datas"
```

```
In [3]: """
Model & dataset setups.
"""

MODEL_NAME = "xlm-roberta-base"

MAX_LENGTH = 140
BATCH_SIZE = 64
EPOCHS = 10
TRAIN_EPISODES_PER_DATASET = 400
```

```

VALIDATION_EPISODES_PER_DATASET = 100
MYBULLY_TEST_EPISODES = 200
LEARNING_RATE = 1e-5
RANDOM_SEED = 42

N_WAY = 6
N_SHOT = 5
N_QUERY = 5
EPISODES_PER_BATCH = 4

EXCLUDED_CATEGORIES = {"fear"}
CATEGORY_LABELS = ["anger", "disgust", "joy", "sadness", "surprise", "neutral"]
category2id = {label: index for index, label in enumerate(CATEGORY_LABELS)}
id2category = {index: label for label, index in category2id.items()}

NUM_LABELS = len(CATEGORY_LABELS)

```

```

In [4]: """
GPU and reproducibility setup.
"""

NUM_GPUS = min(2, torch.cuda.device_count())

DEVICE = torch.device("cuda:0" if NUM_GPUS else "cpu")

print("Using device:", DEVICE, "| GPUs used:", NUM_GPUS)

USE_AMP = NUM_GPUS > 0
AMP_DTYPE = torch.bfloat16 if USE_AMP and torch.cuda.is_bf16_supported() else torch.float16
if NUM_GPUS:
    torch.backends.cuda.matmul.allow_tf32 = True
    torch.backends.cudnn.allow_tf32 = True
    for gpu_id in range(NUM_GPUS):
        props = torch.cuda.get_device_properties(gpu_id)
        print(f"GPU {gpu_id}: {props.name} ({props.total_memory / 2**30:.1f} GiB)")

def set_seed(seed=RANDOM_SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed()

```

Using device: cuda:0 | GPUs used: 1
GPU 0: NVIDIA A100-SXM4-40GB (39.5 GiB)

```

In [5]: """
Shared label normalization and dataset preparation helpers.
"""

with open(GOEMOTIONS_EMOTIONS_PATH, encoding="utf-8") as f:
    goemotions_labels = [line.strip() for line in f if line.strip()]

with open(GOEMOTIONS_EKMAN_PATH, encoding="utf-8") as f:
    ekman_mapping = json.load(f)

```

```

goemotions_to_category = {
    emotion: category.lower()
    for category, emotions in ekman_mapping.items()
    for emotion in emotions
}
goemotions_to_category["neutral"] = "neutral"

# Dataset-native names are mapped where they have a clear shared category.
# Unmapped labels are reported and omitted instead of silently assigned.
native_label_aliases = {
    "anger": "anger", "angry": "anger", "anger tone": "anger",
    "disgust": "disgust", "disgust tone": "disgust",
    "joy": "joy", "happy": "joy", "happiness": "joy", "happily": "joy",
    "love": "joy", "positive": "joy",
    "sad": "sadness", "sadness": "sadness", "sad tone": "sadness",
    "surprise": "surprise", "surprised": "surprise", "surprise tone": "surprise",
    "neutral": "neutral", "neutral tone": "neutral",
    "fear": "fear", "fear tone": "fear",
}

# Mapping CMED Labels to English Labels

CMED_EMOTION_MAPPING = {
    "ā1³æ·jèāžæ°F": "neutral", "é-āîfèāžèāî": "joy",
    "æ,²ā,·èāžèāî": "sadness", "æ†ææ'èāžèāî": "anger",
    "é0šè"̈́èāžèāî": "surprise", "é0šāž†èāžèāî": "surprise",
    "æ̈́̈́æ†%èāžèāî": "fear", "āžæfjèāžèāî": "disgust",
    "ç-‘ā•̈́èāžèāî": "surprise",
}

CMED_EMOTION_MAPPING = {
    "".join(chr(c) for c in (0x5e73, 0x6de1, 0x8a9e, 0x6c23)): "neutral",
    "".join(chr(c) for c in (0x958b, 0x5fc3, 0x8a9e, 0x8abf)): "joy",
    "".join(chr(c) for c in (0x60b2, 0x50b7, 0x8a9e, 0x8abf)): "sadness",
    "".join(chr(c) for c in (0x61a4, 0x6012, 0x8a9e, 0x8abf)): "anger",
    "".join(chr(c) for c in (0x9a5a, 0x8a1d, 0x8a9e, 0x8abf)): "surprise",
    "".join(chr(c) for c in (0x9a5a, 0x5947, 0x8a9e, 0x8abf)): "surprise",
    "".join(chr(c) for c in (0x6050, 0x61fc, 0x8a9e, 0x8abf)): "fear",
    "".join(chr(c) for c in (0x53ad, 0x60e1, 0x8a9e, 0x8abf)): "disgust",
    "".join(chr(c) for c in (0x7591, 0x554f, 0x8a9e, 0x8abf)): "surprise",
}

def normalize_native_label(label):
    value = str(label).strip()
    return CMED_EMOTION_MAPPING.get(value, native_label_aliases.get(value.lower(), None))

def prepare_frame(frame, text_column, label_column, label_mapper, dataset_name):
    frame = frame[[text_column, label_column]].dropna().copy()
    frame["text"] = frame[text_column].astype(str).str.strip()
    frame["category"] = frame[label_column].map(label_mapper)
    unmapped = sorted(frame.loc[frame["category"].isna(), label_column].astype(str).unique())

    if unmapped:
        print(f"{dataset_name}: unmapped source labels omitted: {unmapped}")

    frame = frame.dropna(subset=["category"]).copy()
    before = len(frame)
    frame = frame[~frame["category"].isin(EXCLUDED_CATEGORIES)].copy()

```

```

print(f"{dataset_name}: removed {before - len(frame)} fear examples")

frame = frame[frame["category"].isin(CATEGORY_LABELS)].copy()

frame["label_id"] = frame["category"].map(category2id).astype(int)
frame["dataset"] = dataset_name

return frame[["text", "category", "label_id", "dataset"]].reset_index(drop=True)

```

```

In [6]: """
Dataset loaders
"""

# GoEmotions

goemotions_raw = {}

for split, path in {"train": GOEMOTIONS_TRAIN_PATH, "validation": GOEMOTIONS_VAL
goemotions_raw[split] = pd.read_csv(path, sep="\t", header=None, names=["text", "category", "label_id", "dataset"])

def map_goemotions_label(value):
    first_index = int(str(value).split(",")[0])

    if first_index < 0 or first_index >= len(goemotions_labels):
        return None

    return goemotions_to_category.get(goemotions_labels[first_index])

goemotions_splits = {
    split: prepare_frame(
        df, "text", "emotion", map_goemotions_label, f"GoEmotions/{split}"
    ) for split, df in goemotions_raw.items()
}

```

GoEmotions/train: removed 615 fear examples
GoEmotions/validation: removed 85 fear examples
GoEmotions/test: removed 90 fear examples

```

In [7]: # CMED

cmед_raw = pd.read_csv(CMED_DATA_PATH, usecols=["text", "emotion"])
cmед = prepare_frame(cmед_raw, "text", "emotion", normalize_native_label, "CMED")

cmед_train, cmед_remaining = train_test_split(cmед, test_size=0.2, random_state=42)
cmед_test, cmед_valid = train_test_split(cmед_remaining, test_size=0.5, random_state=42)

cmед_splits = {
    "train": cmед_train.reset_index(drop=True),
    "validation": cmед_valid.reset_index(drop=True),
    "test": cmед_test.reset_index(drop=True)
}

```

CMED: unmapped source labels omitted: ['關切語調']
CMED: removed 0 fear examples

```

In [8]: # MYBully

mybully_valid_raw = pd.read_csv(MYBULLY_VALID_PATH, usecols=["Tweet", "Emotion"])
mybully_test = prepare_frame(mybully_valid_raw, "Tweet", "Emotion", normalize_native_label, "MYBully")

```

MYBully/final_test: removed 19 fear examples

In [9]: *# Dataset analysis*

```
dataset_splits = {"GoEmotions": goemotions_splits, "CMED": cmed_splits}

for dataset_name, splits in dataset_splits.items():
    print(f"\n{dataset_name} sizes:", {split: len(frame) for split, frame in splits.items()})
    print(splits["train"]["category"].value_counts().reindex(CATEGORY_LABELS, fill_value=0))

print("\nMYBully final-test examples:", len(mybully_test))
print(mybully_test["category"].value_counts().reindex(CATEGORY_LABELS, fill_value=0))
```

GoEmotions sizes: {'train': 42795, 'validation': 5341, 'test': 5337}

category

anger 5336

disgust 580

joy 16948

sadness 2619

surprise 4489

neutral 12823

Name: count, dtype: int64

CMED sizes: {'train': 2879, 'validation': 360, 'test': 360}

category

anger 422

disgust 323

joy 473

sadness 389

surprise 708

neutral 564

Name: count, dtype: int64

MYBully final-test examples: 1152

category

anger 620

disgust 83

joy 113

sadness 69

surprise 27

neutral 240

Name: count, dtype: int64

In [10]:

```
"""
Tokenizer and episodic sampling configuration.
"""

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, timeout=120)

TOKEN_CACHE = {}

def sample_episode(frame, n_way=N_WAY, n_shot=N_SHOT, n_query=N_QUERY):
    """Sample a balanced episode and remap its classes to local episode IDs."""
    available = []

    for label_id, group in frame.groupby("label_id"):
        if len(group) >= n_shot + n_query:
            available.append((int(label_id), group))

    episode_n_way = min(n_way, len(available))
```

```

if episode_n_way < 2:
    raise ValueError("An episode needs at least two eligible classes.")

chosen = random.sample(available, episode_n_way)
support_rows, query_rows = [], []

for local_id, (_, group) in enumerate(chosen):
    rows = group.sample(n=n_shot + n_query, random_state=random.randint(0, 2
    support_rows.extend((str(row.text), local_id) for row in rows.iloc[:n_sh
    query_rows.extend((str(row.text), local_id) for row in rows.iloc[n_shot:

episode_category_ids = [label_id for label_id, _ in chosen]
return support_rows, query_rows, episode_category_ids

def encode_texts(texts):
    missing = list(dict.fromkeys(text for text in texts if text not in TOKEN_CAC
    if missing:
        new_tokens = tokenizer(missing, max_length=MAX_LENGTH, truncation=True,
        for text, ids, mask in zip(missing, new_tokens["input_ids"], new_tokens[
            TOKEN_CACHE[text] = {"input_ids": ids, "attention_mask": mask}

    encoded = tokenizer.pad(
        [TOKEN_CACHE[text] for text in texts],
        padding=True, pad_to_multiple_of=8, return_tensors="pt",
    )
    return {key: value.to(DEVICE) for key, value in encoded.items()}

print("Episode shape:", f"{N_WAY}-way, {N_SHOT}-shot, {N_QUERY}-query per class"

```

Episode shape: 6-way, 5-shot, 5-query per class

```

In [11]: """
Prototype network and episode-level objective.
"""

class PrototypeNetwork(torch.nn.Module):
    def __init__(self, model_name):
        super().__init__()
        self.encoder = AutoModel.from_pretrained(model_name)
        self.dropout = torch.nn.Dropout(0.1)

    def forward(self, input_ids, attention_mask):
        with torch.cuda.amp.autocast(enabled=USE_AMP, dtype=AMP_DTYPE):
            outputs = self.encoder(input_ids=input_ids, attention_mask=attention

            token_embeddings = outputs.last_hidden_state.float()
            mask = attention_mask.unsqueeze(-1).to(token_embeddings.dtype)
            mean_embedding = (token_embeddings * mask).sum(dim=1) / mask.sum(dim=1).
            dropped_embedding = self.dropout(mean_embedding)
            return torch.nn.functional.normalize(dropped_embedding, p=2, dim=1)

def load_and_average_encoder_checkpoints(encoder, checkpoint_paths):
    """Average matching XLM-R encoder weights from the two classifier checkpoint
    target_state = encoder.state_dict()
    source_states = []

    for checkpoint_path in checkpoint_paths:

```

```

checkpoint = torch.load(checkpoint_path, map_location="cpu")

if isinstance(checkpoint, dict) and "state_dict" in checkpoint:
    checkpoint = checkpoint["state_dict"]

encoder_state = {}
for key, value in checkpoint.items():
    clean_key = key[7:] if key.startswith("module.") else key

    if clean_key.startswith("roberta."):
        clean_key = clean_key[len("roberta."):]

    if clean_key in target_state and target_state[clean_key].shape == va
        encoder_state[clean_key] = value

coverage = len(encoder_state) / max(1, len(target_state))
if coverage < 0.9:
    raise ValueError(
        f"Checkpoint {checkpoint_path} matched only {coverage:.1%} of en
    )

print(f"Loaded {len(encoder_state)} encoder tensors from {checkpoint_pat
source_states.append(encoder_state)

shared_keys = set(source_states[0]) & set(source_states[1])
shared_coverage = len(shared_keys) / max(1, len(target_state))

if shared_coverage < 0.9:
    raise ValueError(f"The two checkpoints share only {shared_coverage:.1%}

averaged_state = dict(target_state)
for key in shared_keys:
    first, second = source_states[0][key], source_states[1][key]
    if first.is_floating_point():
        averaged_state[key] = (first.float() + second.float()).div(2).to(fir
    else:
        averaged_state[key] = first

encoder.load_state_dict(averaged_state, strict=True)

def initialize_encoder(model):
    base_model = model.module if isinstance(model, torch.nn.DataParallel) else m

    for path in (GOEMOTIONS_CHECKPOINT, CMED_CHECKPOINT):
        if not os.path.isfile(path):
            raise FileNotFoundError(f"Required initialization checkpoint not fou

load_and_average_encoder_checkpoints(
    base_model.encoder, [GOEMOTIONS_CHECKPOINT, CMED_CHECKPOINT]
)

def prototypical_loss(model, support_rows, query_rows):
    support_texts, support_labels = zip(*support_rows)
    query_texts, query_labels = zip(*query_rows)

    all_inputs = encode_texts(list(support_texts) + list(query_texts))
    support_count = len(support_texts)
    support_targets = torch.tensor(support_labels, dtype=torch.long, device=DEVI
    query_targets = torch.tensor(query_labels, dtype=torch.long, device=DEVICE)

```

```

embeddings = model(**all_inputs)
support_embeddings = embeddings[:support_count]
query_embeddings = embeddings[support_count:]
episode_n_way = int(torch.cat([support_targets, query_targets]).max().item())
prototypes = torch.stack([
    support_embeddings[support_targets == class_id].mean(dim=0)
    for class_id in range(episode_n_way)
])

# Cosine distance between each query embedding and class prototype.
normalized_queries = torch.nn.functional.normalize(query_embeddings, p=2, dim=1)
normalized_prototypes = torch.nn.functional.normalize(prototypes, p=2, dim=1)
distances = 1 - normalized_queries @ normalized_prototypes.T
logits = -distances
loss = torch.nn.functional.cross_entropy(logits, query_targets)
predictions = logits.argmax(dim=1)
accuracy = (predictions == query_targets).float().mean()
return loss, accuracy, predictions, query_targets

def batched_prototypical_loss(model, episode_batch):
    """Encode several episodes together to keep both GPUs occupied."""
    all_texts, layouts = [], []

    for support_rows, query_rows, _ in episode_batch:
        start = len(all_texts)
        support_texts, support_labels = zip(*support_rows)
        query_texts, query_labels = zip(*query_rows)

        all_texts.extend(support_texts)
        all_texts.extend(query_texts)

        layouts.append((
            start, len(support_texts),
            torch.tensor(support_labels, dtype=torch.long, device=DEVICE),
            torch.tensor(query_labels, dtype=torch.long, device=DEVICE),
        ))

    embeddings = model(**encode_texts(all_texts))
    losses, accuracies = [], []

    for start, support_count, support_targets, query_targets in layouts:
        query_start = start + support_count
        query_count = len(query_targets)

        support_embeddings = embeddings[start:query_start]
        query_embeddings = embeddings[query_start:query_start + query_count]

        n_way = int(torch.cat([support_targets, query_targets]).max().item()) + 1
        prototypes = torch.stack([
            support_embeddings[support_targets == class_id].mean(dim=0)
            for class_id in range(n_way)
        ])

        # Cosine distance between each query embedding and class prototype.
        normalized_queries = torch.nn.functional.normalize(query_embeddings, p=2, dim=1)
        normalized_prototypes = torch.nn.functional.normalize(prototypes, p=2, dim=1)
        distances = 1 - normalized_queries @ normalized_prototypes.T
        logits = -distances
        losses.append(torch.nn.functional.cross_entropy(logits, query_targets))
        accuracies.append((logits.argmax(dim=1) == query_targets).float().mean())

```

```

        return torch.stack(losses).mean(), torch.stack(accuracies).mean()

model = PrototypeNetwork(MODEL_NAME).to(DEVICE)
initialize_encoder(model)
if NUM_GPUS > 1:
    model = torch.nn.DataParallel(model, device_ids=list(range(NUM_GPUS)))

optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
GRAD_SCALER = torch.cuda.amp.GradScaler(enabled=USE_AMP and AMP_DTYPE == torch.f

print("Prototype encoder initialized on", DEVICE)

```

Loaded 197 encoder tensors from /home/jtan/myssp/models/best_goemotions_focal_model.pt

Loaded 197 encoder tensors from /home/jtan/myssp/models/best_cmed_focal_model.pt
 Prototype encoder initialized on cuda:0

/tmp/ipykernel_2214214/586263585.py:157: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.

GRAD_SCALER = torch.cuda.amp.GradScaler(enabled=USE_AMP and AMP_DTYPE == torch.float16)

In [12]:

```

"""
Episodic training and validation across all datasets.
"""

def run_episodes(model, optimizer=None, episodes_per_dataset=TRAIN_EPISODES_PER_
    training = optimizer is not None
    model.train(training)
    episode_losses, episode_accuracies = [], []

    for dataset_name, splits in dataset_splits.items():
        frame = splits["train" if training else "validation"]

        for start in range(0, episodes_per_dataset, EPISODES_PER_BATCH):
            episode_batch = [
                sample_episode(frame)
                for _ in range(min(EPISODES_PER_BATCH, episodes_per_dataset - st
            ]

            if training:
                optimizer.zero_grad(set_to_none=True)
                loss, accuracy = batched_prototypical_loss(model, episode_batch)
                GRAD_SCALER.scale(loss).backward()
                GRAD_SCALER.unscale_(optimizer)
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                GRAD_SCALER.step(optimizer)
                GRAD_SCALER.update()
            else:
                with torch.no_grad():
                    loss, accuracy = batched_prototypical_loss(model, episode_ba

            episode_losses.append(float(loss.detach().cpu()))
            episode_accuracies.append(float(accuracy.detach().cpu()))

    return {
        "loss": float(np.mean(episode_losses)),
        "accuracy": float(np.mean(episode_accuracies)),
    }

```

```

    }

    best_validation_accuracy = -1.0
    best_model_path = str(MODELS_DIR / "best_prototypical_meta_model.pt")

    for epoch in range(1, EPOCHS + 1):
        train_metrics = run_episodes(
            model, optimizer, TRAIN_EPISODES_PER_DATASET
        )
        validation_metrics = run_episodes(
            model, None, VALIDATION_EPISODES_PER_DATASET
        )

        print(
            f"Epoch {epoch}/{EPOCHS} | "
            f"train loss {train_metrics['loss']:.4f}, "
            f"train episode accuracy {train_metrics['accuracy']:.4f} | "
            f"validation loss {validation_metrics['loss']:.4f}, "
            f"validation episode accuracy {validation_metrics['accuracy']:.4f}"
        )

        if validation_metrics["accuracy"] > best_validation_accuracy:
            best_validation_accuracy = validation_metrics["accuracy"]
            base_model = model.module if isinstance(model, torch.nn.DataParallel) else model
            torch.save(base_model.state_dict(), best_model_path)
            print("Saved improved model to", best_model_path)

```

You're using a XLMRobertaTokenizerFast tokenizer. Please note that with a fast tokenizer, using the `__call__` method is faster than using a method to encode the text followed by a call to the `pad` method to get a padded encoding.

/tmp/ipykernel_2214214/586263585.py:12: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
 with torch.cuda.amp.autocast(enabled=USE_AMP, dtype=AMP_DTYPE):

```

Epoch 1/10 | train loss 1.3188, train episode accuracy 0.6777 | validation loss 1.2679, validation episode accuracy 0.6945
Saved improved model to /home/jtan/myssp/models/best_prototypical_meta_model.pt
Epoch 2/10 | train loss 1.2338, train episode accuracy 0.7405 | validation loss 1.2271, validation episode accuracy 0.7360
Saved improved model to /home/jtan/myssp/models/best_prototypical_meta_model.pt
Epoch 3/10 | train loss 1.1973, train episode accuracy 0.7841 | validation loss 1.2156, validation episode accuracy 0.7522
Saved improved model to /home/jtan/myssp/models/best_prototypical_meta_model.pt
Epoch 4/10 | train loss 1.1699, train episode accuracy 0.8149 | validation loss 1.2090, validation episode accuracy 0.7522
Saved improved model to /home/jtan/myssp/models/best_prototypical_meta_model.pt
Epoch 5/10 | train loss 1.1541, train episode accuracy 0.8286 | validation loss 1.2017, validation episode accuracy 0.7593
Saved improved model to /home/jtan/myssp/models/best_prototypical_meta_model.pt
Epoch 6/10 | train loss 1.1384, train episode accuracy 0.8441 | validation loss 1.2056, validation episode accuracy 0.7578
Epoch 7/10 | train loss 1.1314, train episode accuracy 0.8486 | validation loss 1.1965, validation episode accuracy 0.7623
Saved improved model to /home/jtan/myssp/models/best_prototypical_meta_model.pt
Epoch 8/10 | train loss 1.1222, train episode accuracy 0.8556 | validation loss 1.2044, validation episode accuracy 0.7660
Saved improved model to /home/jtan/myssp/models/best_prototypical_meta_model.pt
Epoch 9/10 | train loss 1.1140, train episode accuracy 0.8664 | validation loss 1.2153, validation episode accuracy 0.7570

```

Epoch 10/10 | train loss 1.1061, train episode accuracy 0.8729 | validation loss 1.2129, validation episode accuracy 0.7590

```
In [13]: """Held-out test episodes and per-dataset classification reports."""

from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix,
)

# Load the checkpoint selected by validation episode accuracy.
model_state = torch.load(best_model_path, map_location=DEVICE)
base_model = model.module if isinstance(model, torch.nn.DataParallel) else model
base_model.load_state_dict(model_state)

def evaluate_test_episodes(model, frame, n_episodes=VALIDATION_EPISODES_PER_DATA
    model.eval()
    actual_labels, predicted_labels = [], []

    with torch.no_grad():
        for _ in range(n_episodes):
            support_rows, query_rows, episode_category_ids = sample_episode(frame
            _, _, predictions, targets = prototypical_loss(
                model, support_rows, query_rows
            )

            # Convert episode-local IDs back to the shared emotion IDs.
            local_predictions = predictions.cpu().tolist()
            local_targets = targets.cpu().tolist()
            predicted_labels.extend(episode_category_ids[i] for i in local_predi
            actual_labels.extend(episode_category_ids[i] for i in local_targets)

    return actual_labels, predicted_labels

for dataset_name, test_frame in [("MYBully", mybully_test)]:
    y_true, y_pred = evaluate_test_episodes(
        model, test_frame, MYBULLY_TEST_EPISODES
    )

    observed_ids = sorted(set(y_true) | set(y_pred))
    observed_names = [id2category[label_id] for label_id in observed_ids]

    print(f"\n=====")
    print(f"{dataset_name.upper()} HELD-OUT EPISODE RESULTS")
    print(f"=====")
    print(f"Accuracy:      {accuracy_score(y_true, y_pred):.4f}")
    print(f"Balanced accuracy: {balanced_accuracy_score(y_true, y_pred):.4f}")
    print(f"Macro-F1:      {f1_score(y_true, y_pred, labels=observed_ids, average=
    print("\nClassification Report:")
    print(classification_report(
        y_true, y_pred, labels=observed_ids, target_names=observed_names,
        digits=4, zero_division=0
    ))

    matrix = confusion_matrix(y_true, y_pred, labels=observed_ids)
    print("Confusion Matrix:")
    print(pd.DataFrame(matrix, index=observed_names, columns=observed_names))
```

```
/tmp/ipykernel_2214214/586263585.py:12: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.  
  with torch.cuda.amp.autocast(enabled=USE_AMP, dtype=AMP_DTYPE):
```

```
=====
MYBULLY HELD-OUT EPISODE RESULTS
=====
```

```
Accuracy:      0.3588
Balanced accuracy: 0.3588
Macro-F1:      0.3444
```

Classification Report:

	precision	recall	f1-score	support
anger	0.3193	0.3720	0.3436	1000
disgust	0.2259	0.1710	0.1946	1000
joy	0.4638	0.4930	0.4779	1000
sadness	0.4335	0.6100	0.5069	1000
surprise	0.2704	0.1560	0.1978	1000
neutral	0.3404	0.3510	0.3456	1000
accuracy			0.3588	6000
macro avg	0.3422	0.3588	0.3444	6000
weighted avg	0.3422	0.3588	0.3444	6000

Confusion Matrix:

	anger	disgust	joy	sadness	surprise	neutral
anger	372	208	63	124	64	169
disgust	314	171	93	193	97	132
joy	105	77	493	91	70	164
sadness	92	80	42	610	113	63
surprise	108	115	194	275	156	152
neutral	174	106	178	114	77	351